ENTERPRISE ARCHITECTURE REFERENCE

Apache Kafka: Technical Guide

What it is, Why it is used, and When to choose it for modern architectures

Topic: Distributed Event Streaming

Target Architecture: Microservices, Real-Time Data & Event-Driven Systems

1 What is Apache Kafka?

Apache Kafka is an open-source, distributed event streaming platform designed for high-throughput, fault-tolerant, and low-latency data pipelines and streaming applications. Originally created by LinkedIn and later open-sourced under the Apache Software Foundation, Kafka is now the industry standard backbone for real-time data across more than 80% of Fortune 100 companies.

> **Core Conceptual Shift:** Unlike traditional message brokers (which treat messages as transient packets deleted immediately after delivery), Kafka acts as a **distributed, replicated, append-only commit log**. Messages are safely persisted on disk and can be replayed as needed.



The 6 Fundamental Concepts

1. Event / Record

A key-value pair with a timestamp and optional headers representing that something happened in the system (e.g., *OrderPlaced*).

2. Topic

A named feed or category where records are published. Analogy: A database table or folder in a filesystem.

3. Partition

The unit of parallelism. Topics are split into ordered, immutable partitions distributed across multiple server brokers.

4. Offset

A sequential ID assigned to each message in a partition. Consumers track their own offset to remember what they have read.

5. Producer & Consumer

Producers publish events. Consumers subscribe to topics. Consumers can belong to a **Consumer Group** for horizontal load sharing.

6. KRaft Consensus

Modern Kafka uses the built-in KRaft (Raft) consensus protocol for cluster metadata, completely eliminating ZooKeeper.

2 Why is Kafka Used? (Key Advantages)

Kafka is chosen over traditional messaging queues (ActiveMQ, RabbitMQ) and standard HTTP APIs because of architectural superpowers tailored for high-scale, modern systems:

⚡ Ultra-High Throughput & Low Latency

Capable of handling millions of records per second with single-digit millisecond latency. It achieves this by using **sequential disk I/O**, OS page caching, and **Zero-Copy** network data transfer (`sendfile`).

🛡️ Fault-Tolerant & Durable Persistence

Data is written to persistent disk and replicated across multiple broker nodes (In-Sync Replicas). If a broker crashes, another replica instantly takes over with zero data loss.

🔄 Massive Decoupling of Services

Producers don't know who consumes their data or how fast. Downstream services can fail, restart, or scale independently without slowing down or crashing the upstream producers.

⏪ Event Replayability & Retention

Unlike traditional queues where messages vanish once read, Kafka retains messages for hours, days, or indefinitely. If a bug occurs, consumers can rewind their offset and reprocess past events.

📈 Linear Horizontal Scalability

Scaling is achieved simply by adding more partitions and adding more brokers or consumer instances. Kafka distributes the workload seamlessly without downtime.

🎯 Guaranteed Strict Ordering

Kafka guarantees strict FIFO ordering within each partition. By assigning a consistent record Key (e.g., `customerId`), all events for that entity are guaranteed to be processed in sequence.

3 When is Kafka Used? (Top Real-World Use Cases)

1. Event-Driven Microservices Architecture

Instead of services calling each other synchronously via REST (creating tight coupling and cascading failure risk), services emit domain events (`OrderCreated`, `PaymentCompleted`) to Kafka. Other services consume and react asynchronously.

2. Real-Time Stream Processing & Analytics

Feeding real-time data into stream processors (Kafka Streams, Apache Flink, Spark Streaming) for instantaneous business value: live fraud detection in banking, dynamic ride pricing (Uber), and real-time recommendations (Netflix).

3. High-Volume Activity & Telemetry Tracking

Collecting millions of user actions (clicks, page views, searches) or IoT sensor telemetries in real time for ingestion into analytics platforms and data warehouses.

4. Change Data Capture (CDC) & Data Pipelines

Using tools like Debezium + Kafka to capture every database insert/update from transactional DBs (PostgreSQL, MySQL, SQL Server) and sync them into search indexes (Elasticsearch) and data lakes (Snowflake, BigQuery).

5. Centralized Log & Metrics Aggregation

Serving as the high-speed ingest buffer for distributed application logs and metrics before piping them into Elasticsearch, Grafana Loki, or Splunk.

4 When NOT to Use Kafka (Decision Matrix)

Kafka is powerful, but it is not a silver bullet. Choosing Kafka when it is not needed introduces unnecessary infrastructure complexity.

Architecture Need	Recommended Technology	Why Kafka is / isn't the right choice
Simple Task Queuing (e.g. background emails, PDF generation)	**RabbitMQ / Celery / Redis**	Kafka lacks per-message acknowledgment, complex routing exchanges, and message-level retries out of the box.
Direct Request-Reply (RPC) (e.g. User Login, Fetching profile data)	**REST / gRPC**	Synchronous request-response is unnatural in Kafka; direct RPC APIs are far simpler and lower overhead.
High-Volume Event Streaming & Replay (e.g. Activity feeds, CDC, Microservices)	**Apache Kafka**	**Best fit:** Petabyte scale, partition-based ordering, multi-consumer durability, replayability.
Complex AMQP Routing Keys (e.g. dynamic routing to specific workers)	**RabbitMQ**	RabbitMQ excels at complex exchange-to-queue bindings and fine-grained routing logic.

✅ USE KAFKA WHEN:

- You have multiple independent consumers for the same data.
- You need to store and replay historical events.
- Throughput exceeds tens of thousands of messages/sec.
- You are building event-driven microservices.

⚠️ AVOID KAFKA WHEN:

- You need individual message acknowledgments or dead-letter retries at the per-message level.
- Low traffic volume (< 100 messages/sec) where operational overhead of Kafka isn't justified.
- Simple synchronous request-reply scenarios.

5 Summary Checklist

- **What:** A distributed append-only log platform for streaming events reliably across services.
- **Why:** Massive throughput, horizontal scalability, fault-tolerant persistence, and complete producer/consumer decoupling.
- **When:** Real-time streaming, event-driven architectures, CDC pipelines, and telemetry at scale.